ERC-20

ERC-20 est un standard qui définit les règles que doit respecter un token fongible sur Ethereum.

Tu peux retenir trois mots :

- ERC : Ethereum Request for Comments.
- 20 : le numéro de ce standard.
- Standard : un ensemble de règles communes que les développeurs suivent.

Mais attention : ERC-20 n'est ni une cryptomonnaie, ni un langage de programmation. C'est une spécification qui explique comment un token doit se comporter.

Par exemple, Alice envoie 10 FT42 à Bob :

- Alice passe de 100 à 90 FT42.
- Bob passe de 20 à 30 FT42.

Le contrat met à jour les soldes. Si Alice n'a pas suffisamment de tokens, le transfert échoue.

Important : un token n'a pas automatiquement une valeur en euros. Sa valeur dépend de son usage et, éventuellement, du marché.

difference 😀

ERC-20 (Tokens fongibles) : Les unités sont interchangeables. Par exemple, 1 FT42 est équivalent à un autre FT42 du même token.

ERC-721 (Tokens non fongibles NFT) : Chaque token représente un élément distinct, comme un objet de collection numérique ou un billet numérique unique.

OpenZeppelin : projet qui fournit des bibliotheques de code : ERC-20 , ERC-721 ....

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract ybouzafoToken42 is ERC20 { --- >Hériter du contrat ERC20

Créer les tokens

```
    _mint(msg.sender, 1000 * 10 ** decimals());
```

C'est ici que les tokens sont créés.

- `_mint` : fonction interne fournie par OpenZeppelin pour créer des tokens et augmenter le solde d'une adresse.

- `msg.sender` : l'adresse qui appelle le constructeur au déploiement ; ici, c'est l'adresse du déployeur.

on herite de la classe de openZepplin , et on peux aussi Créer des tokens supplémentaires après le déploiement, c;est le cas `ERC20` avec une fonction `mint` protégée par des permissions.

Ethereum

Ethereum est un réseau informatique décentralisé. Les ordinateurs qui le font fonctionner vérifient les transactions et maintiennent un registre partagé appelé blockchain.

C’est le réseau informatique qui permet de faire fonctionner des applications, des contrats intelligents et des transactions.

ETH (Ether)

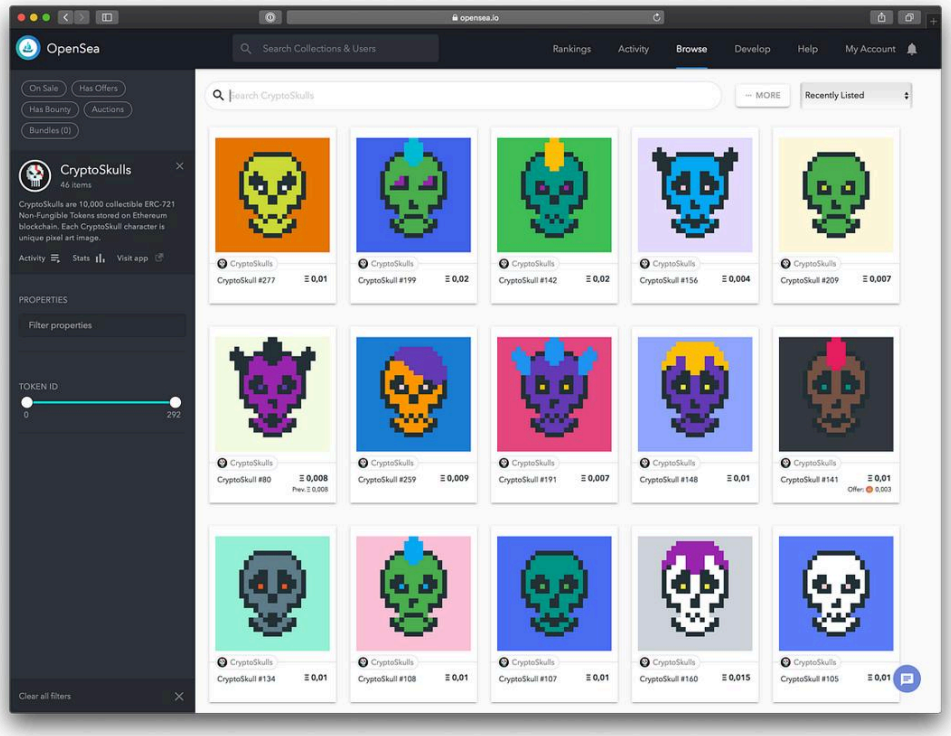
C’est la cryptomonnaie native du réseau. Elle sert notamment à payer les frais de transaction et peut aussi être échangée ou conservée comme actif numérique.

-->

Ethereum ne sert pas seulement à acheter et vendre des cryptomonnaies. Il permet aussi de faire fonctionner plusieurs types d'applications.

La finance décentralisée (DeFi)

Emprunter, prêter ou échanger des actifs numériques grâce à des programmes informatiques, sans passer nécessairement par une banque.



Les NFT et les objets numériques

Créer ou échanger des objets numériques dont la propriété est enregistrée sur la blockchain.

```
contract ERC721 {
    mapping(uint256 id => address owner) public ownerOf;
    mapping(uint256 owner => uint256 balance) public balanceOf;

    event Transfer(address indexed from, address indexed to, uint256 indexed id);

    function mint(address owner, uint256 id) public {
        require(ownerOf[id] == address(0), "already minted");
        ownerOf[id] = owner;
        balanceOf[owner]++;

        emit Transfer(address(0), owner, id);
    }

    function transferFrom(address from, address to, uint256 id) external payable {
        require(ownerOf[id] == msg.sender, "transfer not allowed");
        ownerOf[id] = to;

        balanceOf[from]--;
        balanceOf[to]++;

        emit Transfer(from, to, id);
    }
}
```

Les contrats intelligents (smart contracts)

Des programmes qui exécutent automatiquement des règles prédéfinies lorsque certaines conditions sont remplies.

fonction ERC-20 :

4. `approve(spender, amount)` : autoriser une dépense

C'est une fonction très importante, car elle est souvent mal comprise.

Imagine que tu utilises une application qui doit dépenser des FT42 pour toi. Tu peux autoriser son smart contract à dépenser jusqu'à 100 FT42.

```
approve(adresseDuContrat, 100 * 10 ** 18);
```

Cette fonction ne transfère pas les tokens. Elle enregistre une autorisation.

Par exemple :

- tu possèdes 1 000 FT42 ;
- tu autorises un contrat à dépenser 100 FT42 ;
- tu possèdes toujours 1 000 FT42 ;
- le contrat dispose d'une autorisation pouvant aller jusqu'à 100 FT42.

L'autorisation et le solde sont deux informations différentes.

5. `transferFrom(from, to, amount)` : transférer avec une autorisation

Cette fonction permet à un compte autorisé de transférer des tokens depuis le compte d'un propriétaire.

```
transferFrom(toi, alice, 30 * 10 ** 18);
```

Supposons que tu aies autorisé un smart contract à dépenser 100 FT42.

Si ce contrat appelle `transferFrom` pour envoyer 30 FT42 de ton adresse à Alice :

- ton solde diminue de 30 FT42 ;
- le solde d'Alice augmente de 30 FT42 ;
- l'autorisation restante passe de 100 à 70 FT42, dans le cas habituel d'une autorisation limitée.

À retenir : `approve` autorise, tandis que `transferFrom` utilise cette autorisation pour transférer des tokens.

5. Les événements ERC-20 : `Transfer` et `Approval`

Un contrat ERC-20 standard émet notamment deux événements.

`Transfer`

Signale un transfert de tokens. Cet événement est également utilisé lors de la création ou de la destruction de tokens, avec une adresse nulle comme source ou destination selon le cas.

`Approval`

Signale qu'une autorisation de dépense a été accordée.

Pourquoi sont-ils utiles ?

Les portefeuilles, les explorateurs blockchain et les applications peuvent lire ces événements pour suivre les transferts et les autorisations. Les événements ne remplacent cependant pas l'état du contrat : ils constituent des informations de journalisation.

6. Et `mint` et `burn`, est-ce que ce sont des fonctions ERC-20 ?

Il faut faire attention à cette distinction.

- `mint` signifie créer des tokens.
- `burn` signifie détruire des tokens.

Ce sont des opérations courantes dans les contrats de tokens, mais ERC-20 ne rend pas automatiquement la création et la destruction accessibles au public. Elles dépendent de l'implémentation du contrat.

Dans notre exemple :

```
        _mint(msg.sender, 1000 * 10 ** decimals());
```

Le préfixe `_` est important : dans OpenZeppelin, `_mint` est une fonction interne destinée à être appelée par le code du contrat. Ce n'est pas une fonction publique que tu peux directement cliquer dans Remix.

Notre contrat crée les 1 000 FT42 au déploiement. Il ne propose pas de fonction publique permettant d'en créer d'autres ensuite.

events:

en solodity est une sorte de log , imprimé directement sur la blockchain lorsqu'une action spécifique se produit.

1. Les événements obligatoires du standard ERC-20

Selon la spécification officielle **ERC-20**, votre contrat doit obligatoirement inclure ces deux événements :

- **`event Transfer(address indexed from, address indexed to, uint256 value);`**
  - **Quand est-il émis ?** Chaque fois que des jetons sont transférés d'un compte à un autre, mais aussi lors du **`mint`** (création) et du **`burn`** (destruction).
  - **Détails :**
    - Lors d'un **transfert classique** : `from` = adresse de l'expéditeur, `to` = adresse du destinataire.
    - Lors d'un **`mint`** : `from` = `address(0)` (adresse zéro), car les jetons sont créés à partir de rien.
    - Lors d'un **`burn`** : `to` = `address(0)`, car les jetons sont envoyés à l'adresse zéro pour être détruits.
- **`event Approval(address indexed owner, address indexed spender, uint256 value);`**
  - **Quand est-il émis ?** Lorsque le propriétaire d'un compte autorise un tiers (une `spender` address ou un autre contrat) à dépenser une certaine quantité de jetons en son nom via la fonction `approve`.

2. Les événements personnalisés (Optionnels mais recommandés pour le sujet)

Dans votre contrat, vous avez également ajouté votre propre événement personnalisé pour notifier spécifiquement la création de jetons :

- **`event TokensMinted(address indexed to, uint256 amount);`**
  - **Quand est-il émis ?** Directement dans votre fonction publique `mint` lorsque l'administrateur (`onlyOwner`) génère de nouveaux jetons.

En résumé

1. **Si vous héritez d'OpenZeppelin (`import "@openzeppelin/contracts/token/ERC20/ERC20.sol";`) :**
   - L'événement standard **`Transfer`** est déjà automatiquement émis en arrière-plan lorsque vous appelez la fonction interne `_mint(...)` ou

`_transfer(...)`.

2. **Pour votre fonction `mint` spécifique :**
    - Votre ligne `emit TokensMinted(to, amount);` permet d'ajouter un journal explicite dans la console de Remix et sur Sepolia Etherscan pour prouver au correcteur que l'action de *minting* s'est déroulée avec succès.

sécurité

3]. La sécurité arithmétique (*Overflow / Underflow*)

- **Le principe :** En informatique, une variable numérique a une limite maximale. Si on dépasse cette limite (overflow), le compteur repart à zéro, ce qui peut vider des comptes.
- **Dans votre projet :** Heureusement, à partir de la version **0.8.x de Solidity**, le compilateur intègre une protection automatique (*built-in checks*) contre les dépassements de capacité. Si un calcul dépasse la limite, la transaction est automatiquement annulée au lieu de corrompre les données.

Que la sécurité du contrat repose sur la bibliothèque officielle OpenZeppelin.

Que la création de jetons (minting) est strictement protégée par le mécanisme onlyOwner.

Que le compilateur Solidity moderne protège nativement le contrat contre les erreurs arithmétiques.

LE MÉCANISME DE CONSENSUS:

est exactement le genre de problème que la blockchain doit résoudre : comment un réseau d'ordinateurs peut-il se mettre d'accord sur les transactions à enregistrer, sans dépendre d'une seule personne qui décide de tout ?

Le consensus est la méthode qui permet au réseau de décider quels blocs sont acceptés dans la blockchain.

Décortiquons les mots :

- Mécanisme : une méthode, des règles.
- Consensus : un accord.
- Blockchain : un registre partagé entre de nombreux ordinateurs.

A. Proof of Work (PoW) — Preuve de travail

Exemple : Bitcoin

Imagine une compétition entre des ordinateurs. Ils effectuent énormément de calculs pour trouver une solution qui respecte les règles du réseau.

1. Les mineurs essaient de trouver une solution valide.
2. Un mineur trouve une solution et propose un bloc.
3. Les autres participants vérifient la solution et le bloc.
4. Le réseau suit les règles de la chaîne valide ayant le plus de travail cumulé.

Le but est de rendre la manipulation de l'historique coûteuse en calcul et en énergie.

B. Proof of Stake (PoS) — Preuve d'enjeu

Exemple : Ethereum aujourd'hui

Ici, le fonctionnement est différent : les participants appelés validateurs immobilisent des ETH pour participer à la validation et à la sécurisation du réseau.

1. Des validateurs sont sélectionnés selon le protocole pour proposer ou attester des blocs.
2. Les validateurs vérifient les blocs et participent au consensus.
3. Le protocole détermine quels blocs sont acceptés et finalisés.
4. Des pénalités peuvent s'appliquer aux validateurs qui enfreignent certaines règles.

Il n'est donc pas nécessaire de faire la compétition de calculs intensive utilisée par le Proof of Work.

5. Quelle est la différence entre PoW et PoS ?

	Proof of Work	**Proof of Stake**
Participants principaux	Mineurs	Validateurs
Ressource utilisée	Puissance de calcul	ETH mis en jeu
Exemple	Bitcoin	Ethereum
Sécurité	Coût du travail de calcul	Mise en jeu et pénalités pr protocole

Important pour toi : Ethereum utilisait autrefois le Proof of Work, mais depuis septembre 2022, il utilise le Proof of Stake.

voici ce qui se passe :

1. Tu demandes au contrat de transférer 10 FT42.
2. Le contrat vérifie les conditions, par exemple ton solde.
3. La transaction est diffusée au réseau Ethereum.
4. Les validateurs participent au mécanisme de consensus d'Ethereum.
5. Lorsque la transaction est incluse dans un bloc accepté, le transfert est exécuté dans l'état de la blockchain.